

모델 성능 개선을 위한 여정: 시간 정보의 올바른 활용법 탐색

예측 정확도 향상을 위한 피처 엔지니어링 및 모델 아키텍처 비교 분석

프로젝트의 배경: 왜 전력 수요 예측이 중요한가?

- 전력 수요 예측은 에너지의 효율적 관리와 안정적인 전력 공급을 위해 필수적입니다.
- 특히 서울과 같은 대도시의 경우, 지역별, 시간대별로 전력 사용 패턴이 크게 다르며, 날씨 조건에 따라서도 전력 수요가 큰 영향을 받습니다.
- 본 프로젝트는 서울시 법정동별 전력 사용량 데이터와 기상 데이터를 결합하여, 시간정보의 올바른 활용이 예측 성공에 미치는 영향을 체계적으로 분석하고자 합니다.

목표

날씨 데이터와 시간 데이터를
활용한 전력량 예측 모델 개발

업무 분장

고은채

- 데이터 수집
- 기상데이터 전처리
- EDA
- 머신러닝 모델
- 웹사이트 구현

김동준

- 데이터 수집
- 데이터 정합성 확보
- EDA
- 머신러닝 모델
- 모델 성능 향상 (휴일여부)

김재학

- 데이터 수집
- 기상데이터 전처리
- EDA
- 딥러닝 모델
- 모델성능 향상(휴일 여부)

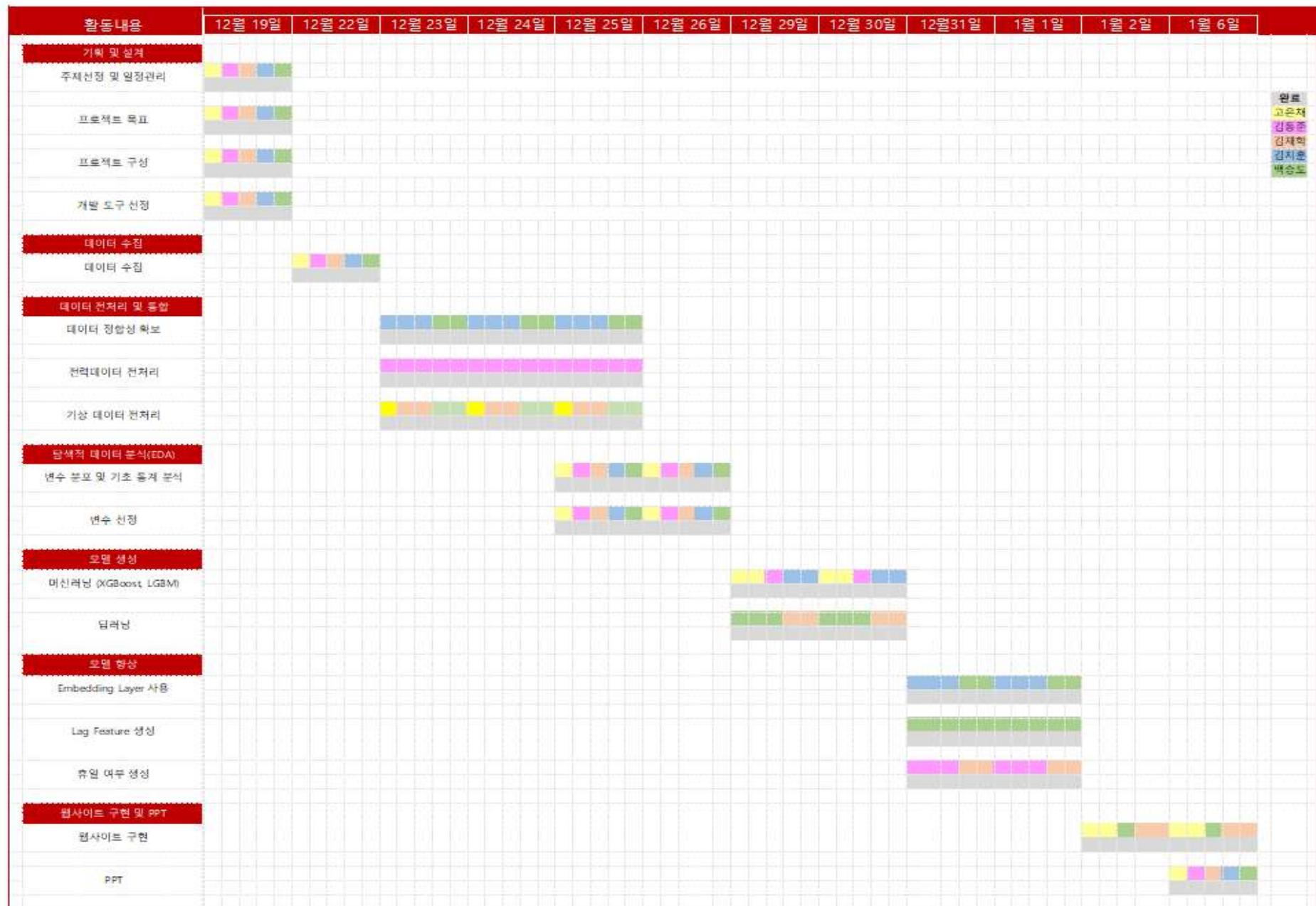
김지훈

- 데이터 수집
- 전력데이터 전처리
- EDA
- 머신러닝 모델
- 모델 성능 향상
(Embedding Layer)

백승도

- 데이터 수집
- 데이터 정합성 확보
- EDA
- 딥러닝 모델
- 모델 성능 향상
(Embedding Layer, LAG)

Gantt Chart



워크 플로우

① 데이터 수집

- 서울시 법정동 전력 사용량 데이터
- 기상청 날씨 데이터

② 데이터 전처리

- 결측치 및 이상치 처리
- 데이터 통합 및 정규화

③ EDA

- 전력 사용 패턴 분석
- 상관관계/정보량 분석
- 다중공산성 분석

④ 모델 생성 (Baseline)

- DNN 기반 베이스라인
- 날씨/전력 데이터 활용
- 초기 성능 측정

⑤ 성능 향상

- 날씨 변수 추가
- LAG/휴일여부 변수 추가
- Embedding Layer 추가

⑥ 결과 분석

- 모델별 성능 분석
- 최적 모델 선정
- 인사이트 도출

데이터

법정동 시간별 전력 사용량

<https://data.seoul.go.kr/dataList/OA-22835/F/1/datasetView.do>

법정동 코드 조회자료

<https://www.code.go.kr/stdcode/regCodeL.do>

서울 기상 관측 데이터

<https://data.kma.go.kr/data/grnd/selectAsosRltnList.do?pgmNo=36&tabNo=1>

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9754804 entries, 0 to 9754803
Data columns (total 5 columns):
#   Column      Dtype
---  -
0   SIGUNGU_CD  int64
1   BJDONG_CD   int64
2   USE_YM      int64
3   USE_HM      int64
4   FDRCT_VLD_KWH float64
dtypes: float64(1), int64(4)
memory usage: 372.1 MB
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1112 entries, 0 to 1111
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   전체코드    1112 non-null  int64
1   법정동명(전체) 1112 non-null  object
2   폐지구분    1112 non-null  object
dtypes: int64(1), object(2)
memory usage: 26.2+ KB
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 59 columns):
#   Column      Non-Null Count  Dtype
---  -
0   지점        365 non-null  int64
1   일시        365 non-null  object
2   평균기온(°C) 365 non-null  float64
3   최저기온(°C) 365 non-null  float64
4   최저기온 시각(hhmm) 365 non-null  int64
5   최고기온(°C) 365 non-null  float64
6   최고기온 시각(hhmm) 365 non-null  int64
7   강수 계속시간(hr) 149 non-null  float64
8   10분 최대 강수량(mm) 190 non-null  float64
9   10분 최대강수량 시각(hhmm) 68 non-null  float64
10  1시간 최대강수량(mm) 100 non-null  float64
11  1시간 최대 강수량 시각(hhmm) 71 non-null  float64
12  일강수량(mm) 150 non-null  float64
13  최대 순간 풍속(m/s) 363 non-null  float64
14  최대 순간 풍속 풍향(16방위) 363 non-null  float64
15  최대 순간풍속 시각(hhmm) 363 non-null  float64
16  최대 풍속(m/s) 363 non-null  float64
17  최대 풍속 풍향(16방위) 363 non-null  float64
18  최대 풍속 시각(hhmm) 363 non-null  float64
19  평균 풍속(m/s) 363 non-null  float64
...
57  9-9강수(mm) 155 non-null  float64
58  안개 계속시간(hr) 8 non-null  float64
dtypes: float64(51), int64(7), object(1)
memory usage: 168.4+ KB
```


데이터 전처리

전력량 데이터

① 중복 데이터 삭제

- 같은 값을 가지는 중복 데이터가 존재
- 중복 데이터들을 삭제하여 처리

② 데이터 선형 보간

- 비어있는 시간대가 존재
- 해당 시간대를 선형 보간으로 값 매꾸기

③ 시간별 -> 일별 데이터로

- 시간대별 전력 사용량을 합쳐 일별 전력 사용량으로 변경

④ 코드 데이터와 합치기

- 지역코드로 되어 있는 것이 보기 어려우니 지역명을 추가

	SIGUNGU_CD	BJDONG_CD	USE_YM	USE_HM	FDRCT_VLD_KWH
0	11650	10700	20220628	100	10782.0565
1	11650	10800	20220628	100	11394.8635
2	11650	10900	20220628	100	7273.9620
3	11740	10300	20220628	100	11008.8110
4	11710	11300	20220628	100	2905.1120



	전체코드	시군구코드	시군구명	법정동코드	법정동명	날짜	파워
0	1111010100	11110	종로구	10100	청운동	2022-12-04	146294.6135
1	1111010100	11110	종로구	10100	청운동	2022-12-05	175633.8270
2	1111010100	11110	종로구	10100	청운동	2022-12-06	156084.1910
3	1111010100	11110	종로구	10100	청운동	2022-12-08	177018.8420
4	1111010100	11110	종로구	10100	청운동	2022-12-09	158467.1690

데이터 전처리

날씨 데이터

① 사용하지 않을 열(변수) 제거

- hhmi 단위를 가지는 열
(ex. 최저기온 시각)
- 동일한 값을 가지는 열 (ex. 지점)

② 결측치 처리

- 대부분이 결측치인 열은 제거
- 강수량의 결측치는 0의 의미
- 나머지는 선형 보간

③ 날씨 데이터 합치기

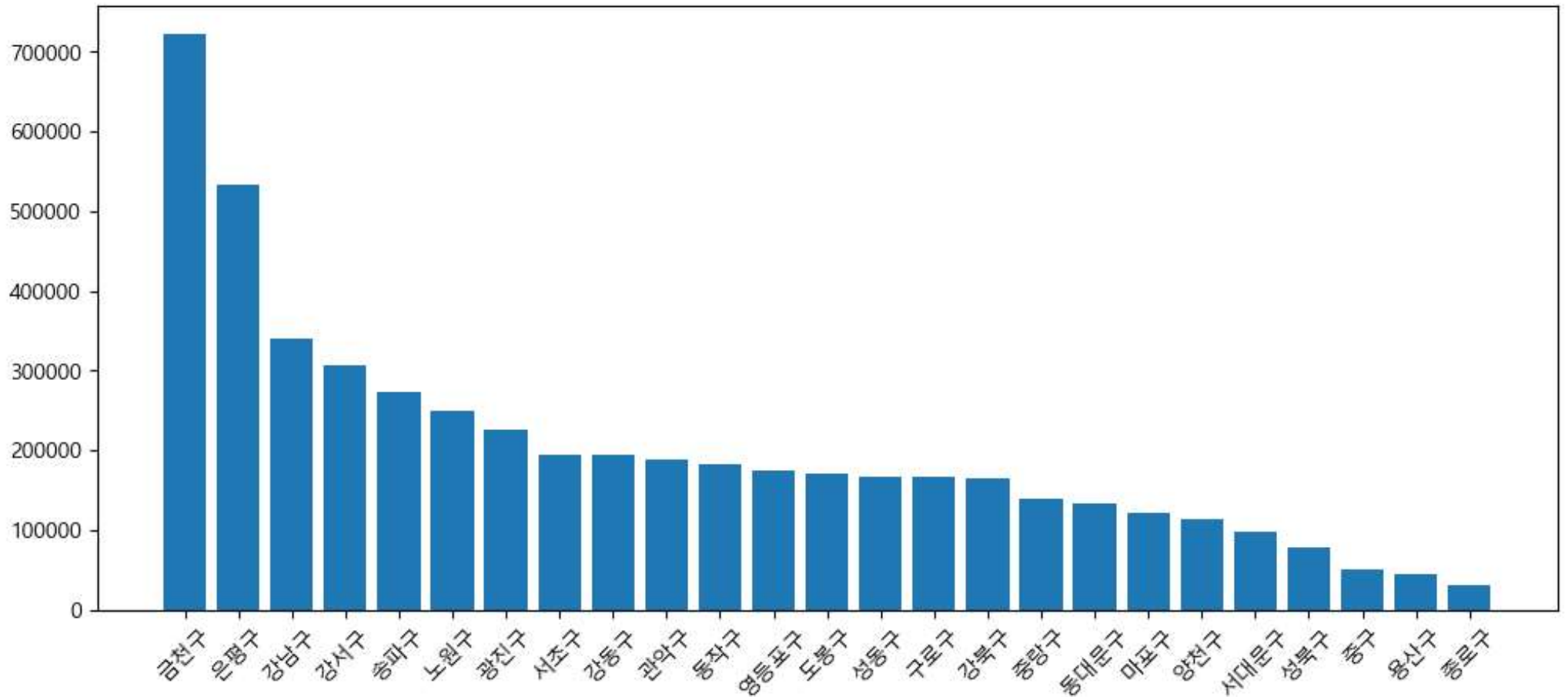
- 날씨 데이터의 경우 1년씩 총 3개의 데이터가 존재하기에 이를 하나의 데이터로 합치기

④ 전력데이터와 합치기

	일시	평균기온(°C)	최저기온(°C)	최고기온(°C)	일강수량(mm)	최대 순간 풍속(m/s)	최대 순간 풍속 풍향(16방향)	최대 풍속 풍향(16방향)	평균 풍속(m/s)	...	평균 20cm 지중온도(°C)	평균 30cm 지중온도(°C)	0.5m 지중온도(°C)	1.0m 지중온도(°C)	1.5m 지중온도(°C)	3.0m 지중온도(°C)	5.0m 지중온도(°C)	합계 대형 증발량(mm)	합계 소형 증발량(mm)	9-9강수(mm)																
1091	2024-12-27	-2.6	-5.9	1.8	0.0	10.4	290.0	5.8	320.0	2.7	...	1.0	2.0	3.7	7.5	10.7	17.3	18.2	1.3	1.8	0.0															
1092	2024-12-28	-3.0	-6.7	1.1	0.0	9.9	290.0	5.8	290.0	2.5	...	0.9	1.9	3.6	7.4	10.5	17.2	18.2	1.3	1.9	0.0															
1093	2024-12-29	1.1	-4.2	6.1	0.0	5.0	180.0	2.8	250.0	1.5	...	0.7	1.7	3.5	7.2	10.4	17.0	18.2	1.3	1.8	0.0															
1094	2024-12-30	5.5	1.9	10.1	0.0	12.0	230.0	6.9	230.0	2.3	...	0.7	1.7	3.4																						
1095	2024-12-31	0.6	-2.0	4.9	0.0	9.6	290.0	5.0	270.0	2.6	...	0.8	1.7	3.4	0	1111010100	11110	종로구	10100	청운동	2022-12-04	146294.6135	-3.5	-5.2	-0.2	...	3.5	5.2	7.5	12.1	15.6	18.4	18.7	1.2	1.7	0.0
5 rows × 39 columns															1	1111010100	11110	종로구	10100	청운동	2022-12-05	175633.8270	-3.1	-7.0	1.9	...	2.8	4.6	7.0	11.7	15.3	18.4	18.7	1.5	2.1	0.1
															2	1111010100	11110	종로구	10100	청운동	2022-12-06	156084.1910	0.9	-3.9	5.3	...	2.4	4.1	6.6	11.3	15.1	18.3	18.7	1.3	1.9	0.1
															3	1111010100	11110	종로구	10100	청운동	2022-12-08	177018.8420	3.7	0.1	9.1	...	3.6	4.8	6.6	10.7	14.5	18.2	18.6	1.5	2.2	0.0
															4	1111010100	11110	종로구	10100	청운동	2022-12-09	158467.1690	4.5	0.9	10.2	...	3.9	4.9	6.6	10.5	14.3	18.1	18.6	1.2	1.8	0.0
															5 rows × 45 columns																					

EDA (지역과 전기 사용량)

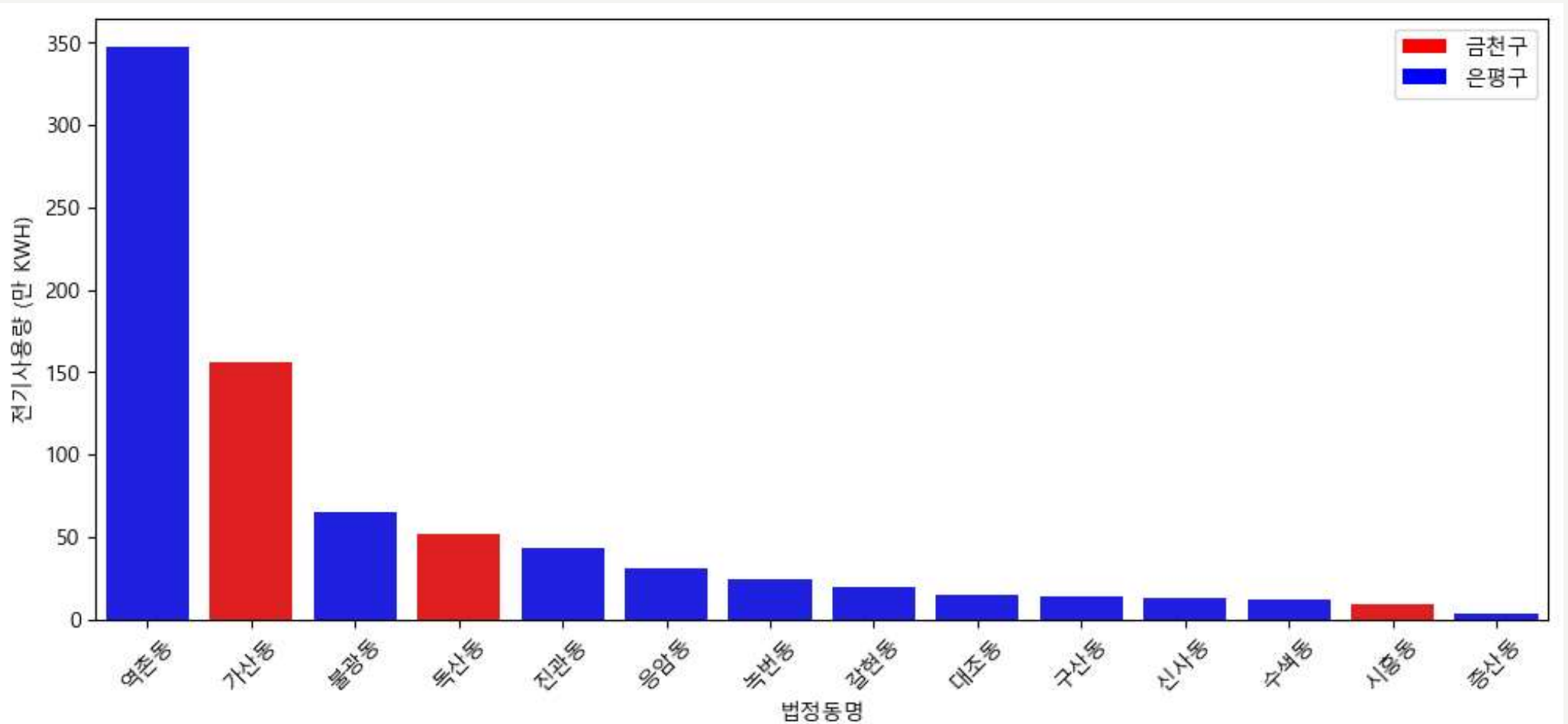
서울시 행정구별 전기 사용량



금천구와 은평구의 전력 사용량이 높은 것을 알 수 있습니다.

EDA (지역과 전기 사용량)

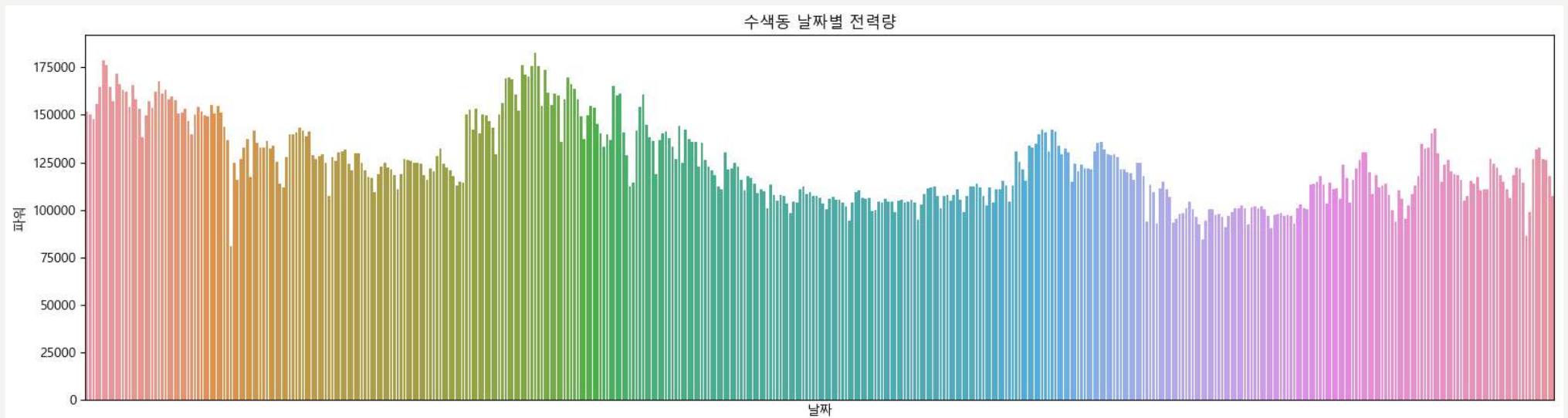
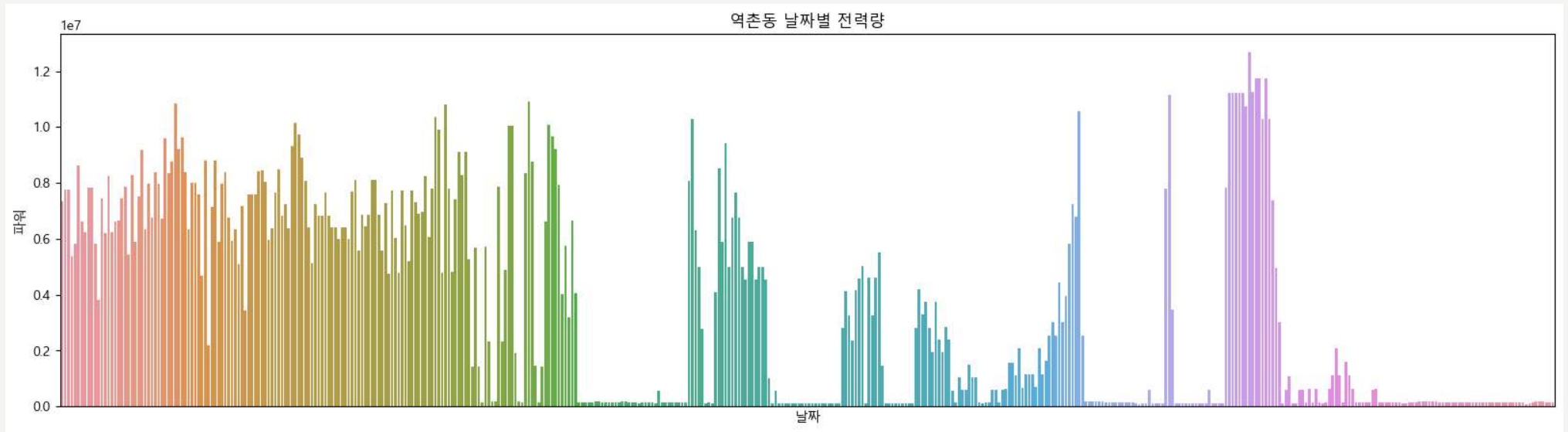
금천구와 은평구 법정동별 전력 사용량



역촌동의 전기사용량이 비정상적으로 높은 것을 볼 수 있습니다.

EDA (지역과 전기 사용량)

역촌동과 수색동의 전기사용량 그래프 비교

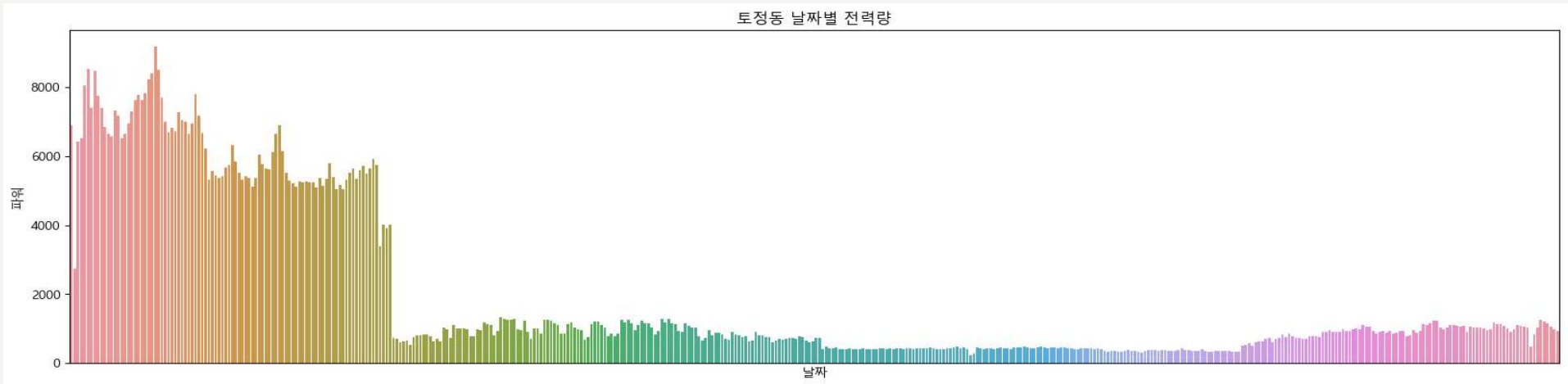


같은 구의 수색동과 비교하였을 때 역촌동의 전력사용량 그래프는 이상하다는 것을 볼 수 있음

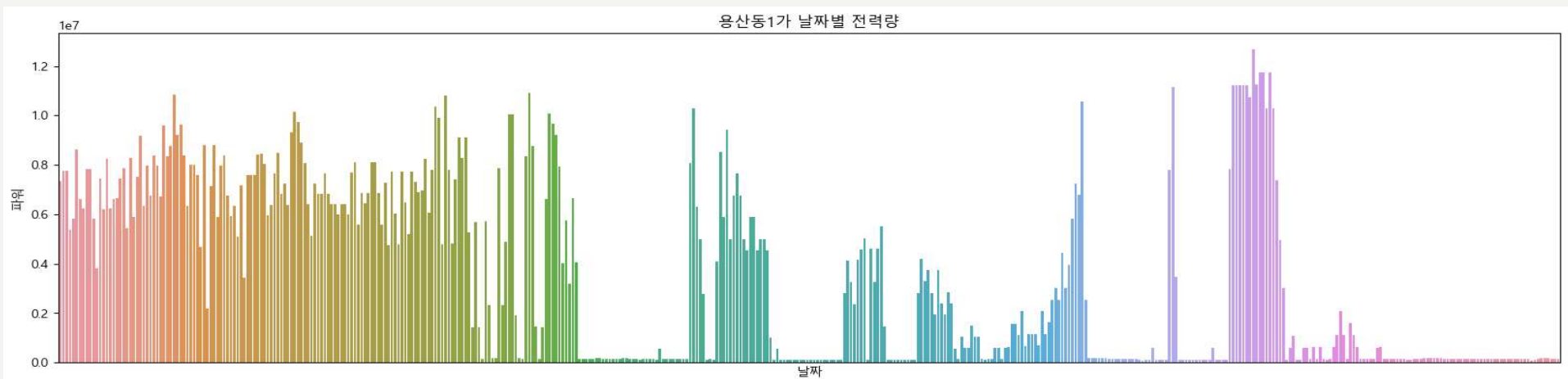
EDA (지역과 전기 사용량)

역촌동과 같은 법정동은 훈련에서 제외하기 위해 법정동의 이상치의 수와 변동계수를 확인

- 이상치가 80개 이상인 토종동



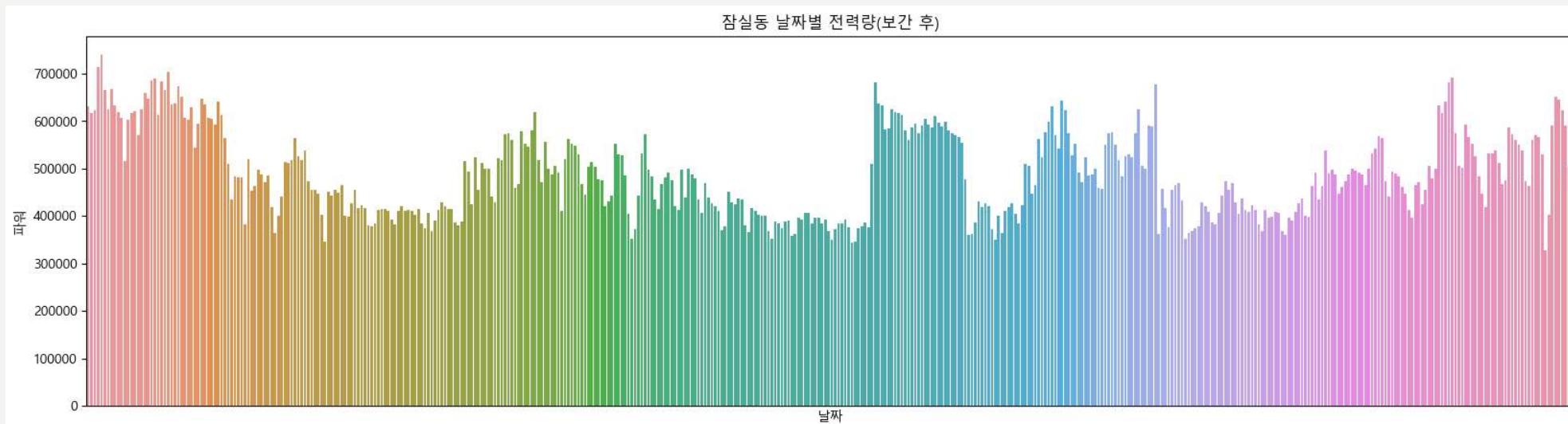
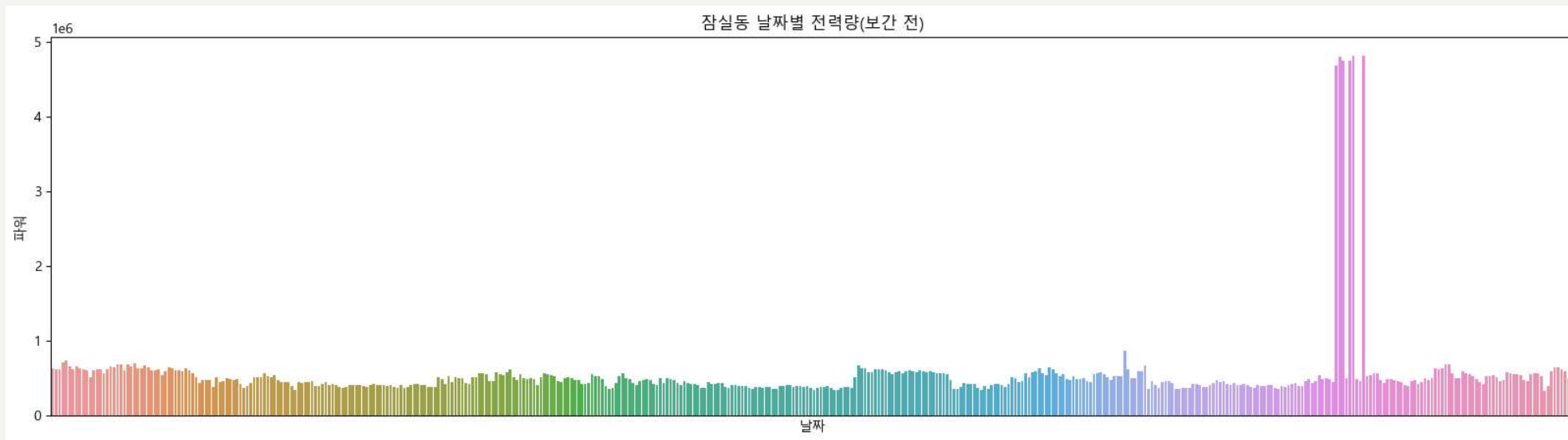
- 변동계수가 큰(1 이상) 용산동 1가, 역촌동 등 10개



총 11개의 법정동을 훈련에서 제외

EDA (지역과 전기 사용량)

각 법정동별 이상치들을 선형 보간을 통해 완화



EDA (변수선택)

- 독립변수와 타겟변수 사이의 상관관계 분석
 - 타겟변수와 가장 상관관계가 높은 것은 지역코드
 - 예상과 다르게 날씨 데이터와 타겟변수간의 상관관계는 0.1을 넘지 못한다
- 날씨의 상관관계가 낮은 이유
 - 지역별로 전력 사용량이 다르기에 전력량과 날씨를 바로 비교하면 안된다
 - 날씨와 전력사용량은 선형관계가 아니다

전체코드	0.520440
시군구코드	0.520117
법정동코드	-0.436198
파워	1.000000
평균기온(°C)	0.038837
최저기온(°C)	0.039129
최고기온(°C)	0.034243
일강수량(mm)	0.021262
평균 이슬점온도(°C)	0.042981
최소 상대습도(%)	0.049505
평균 상대습도(%)	0.038029
평균 증기압(hPa)	0.043046
평균 현지기압(hPa)	-0.026888
최고 해면기압(hPa)	-0.029181
최저 해면기압(hPa)	-0.025407
평균 해면기압(hPa)	-0.027205
평균 지면온도(°C)	0.030868
최저 초상온도(°C)	0.038425
평균 5cm 지중온도(°C)	0.044860
평균 10cm 지중온도(°C)	0.043402
평균 20cm 지중온도(°C)	0.046866
평균 30cm 지중온도(°C)	0.050215
0.5m 지중온도(°C)	0.052383
1.0m 지중온도(°C)	0.056633

EDA (변수선택)

- 랜덤한 법정동의 독립변수와 타겟변수의 상관관계
 - 앞서 보았던 것과 다르게 날씨데이터와 전력사용량사이의 상관관계 계수가 높아진 것을 볼 수 있습니다.

평균기온(°C)	0.038837
최저기온(°C)	0.039129
최고기온(°C)	0.034243
일강수량(mm)	0.021262
평균 이슬점온도(°C)	0.042981
최소 상대습도(%)	0.049505
평균 상대습도(%)	0.038029
평균 증기압(hPa)	0.043046
평균 현지기압(hPa)	-0.026888
최고 해면기압(hPa)	-0.029181
최저 해면기압(hPa)	-0.025407
평균 해면기압(hPa)	-0.027205
평균 지면온도(°C)	0.030868
최저 초상온도(°C)	0.038425



파워	1.000000
평균기온(°C)	0.504035
최저기온(°C)	0.490841
최고기온(°C)	0.487148
일강수량(mm)	0.146033
평균 이슬점온도(°C)	0.477569
최소 상대습도(%)	0.230899
평균 상대습도(%)	0.189606
평균 증기압(hPa)	0.478103
평균 현지기압(hPa)	-0.445844
최고 해면기압(hPa)	-0.467915
최저 해면기압(hPa)	-0.426861
평균 해면기압(hPa)	-0.451484
가조시간(hr)	0.450571

EDA (변수선택)

- 랜덤 법정동의 정보량 분석

	feature	mutual_info
23	평균 30cm 지중온도(°C)	0.708822
22	평균 20cm 지중온도(°C)	0.704405
24	0.5m 지중온도(°C)	0.674614
20	평균 5cm 지중온도(°C)	0.672397
1	최저기온(°C)	0.649296
0	평균기온(°C)	0.637676
21	평균 10cm 지중온도(°C)	0.625596
18	평균 지면온도(°C)	0.612650
19	최저 초상온도(°C)	0.553063
7	평균 증기압(hPa)	0.538143
2	최고기온(°C)	0.519050
4	평균 이슬점온도(°C)	0.514384
28	5.0m 지중온도(°C)	0.491116

25	1.0m 지중온도(°C)	0.478355
26	1.5m 지중온도(°C)	0.431986
12	가조시간(hr)	0.423591
27	3.0m 지중온도(°C)	0.377101
9	최고 해면기압(hPa)	0.345510
11	평균 해면기압(hPa)	0.286327
8	평균 현지기압(hPa)	0.242335
10	최저 해면기압(hPa)	0.190918
5	최소 상대습도(%)	0.109943
6	평균 상대습도(%)	0.107104
14	1시간 최다일사량(MJ/m2)	0.092122
17	평균 중하층운량(1/10)	0.073269
30	합계 소형증발량(mm)	0.070622
3	일강수량(mm)	0.062393

온도와 관련된 값들은 높은 정보량이 나온것이 보이며 다른 값들은 비교적 낮은 정보량이 나온것을 확인

EDA (변수선택)

- 다중공산성

- VIF(분산팽창요인)을 계산하여 독립변수의 다중공산성 확인
- 기상 날씨 데이터들의 VIF 값이 너무 높게 나오는걸 확인

- VIF 값이 너무 높은 이유

- 비슷한 카테고리의 값들이 너무 많아 VIF값이 높게 측정됩니다.
- 이를 해결하기 위해 비슷한 카테고리의 변수들끼리 묶어 카테고리 당 하나의 변수를 선택합니다.

	feature	VIF
0	const	97544.237388
11	평균 해면기압(hPa)	40516.344980
8	평균 현지기압(hPa)	37637.060437
17	평균 20cm 지중온도(°C)	23820.179601
18	평균 30cm 지중온도(°C)	17793.039305
16	평균 10cm 지중온도(°C)	8051.519275
19	0.5m 지중온도(°C)	6101.985643
20	1.0m 지중온도(°C)	5025.973481
21	1.5m 지중온도(°C)	2618.085692
15	평균 5cm 지중온도(°C)	1347.277390
1	평균기온(°C)	1057.278815
4	평균 이슬점온도(°C)	735.408761
2	최저기온(°C)	238.179188
22	3.0m 지중온도(°C)	194.779067
3	최고기온(°C)	184.305025
23	5.0m 지중온도(°C)	127.119037
12	가조시간(hr)	120.190744
13	평균 지면온도(°C)	104.710187
9	최고 해면기압(hPa)	93.336851
10	최저 해면기압(hPa)	88.494312

EDA (변수선택)

- 최종 선택된 변수
 - 각 카테고리 당 상관관계와 정보량 등을 바탕으로 선택된 최종 변수들과 VIF

	feature	VIF
0	const	214.056972
1	최저기온(°C)	13.202527
2	0.5m 지중온도(°C)	8.742195
3	평균 증기압(hPa)	13.075441
4	가조시간(hr)	3.288322
5	평균 상대습도(%)	2.264655

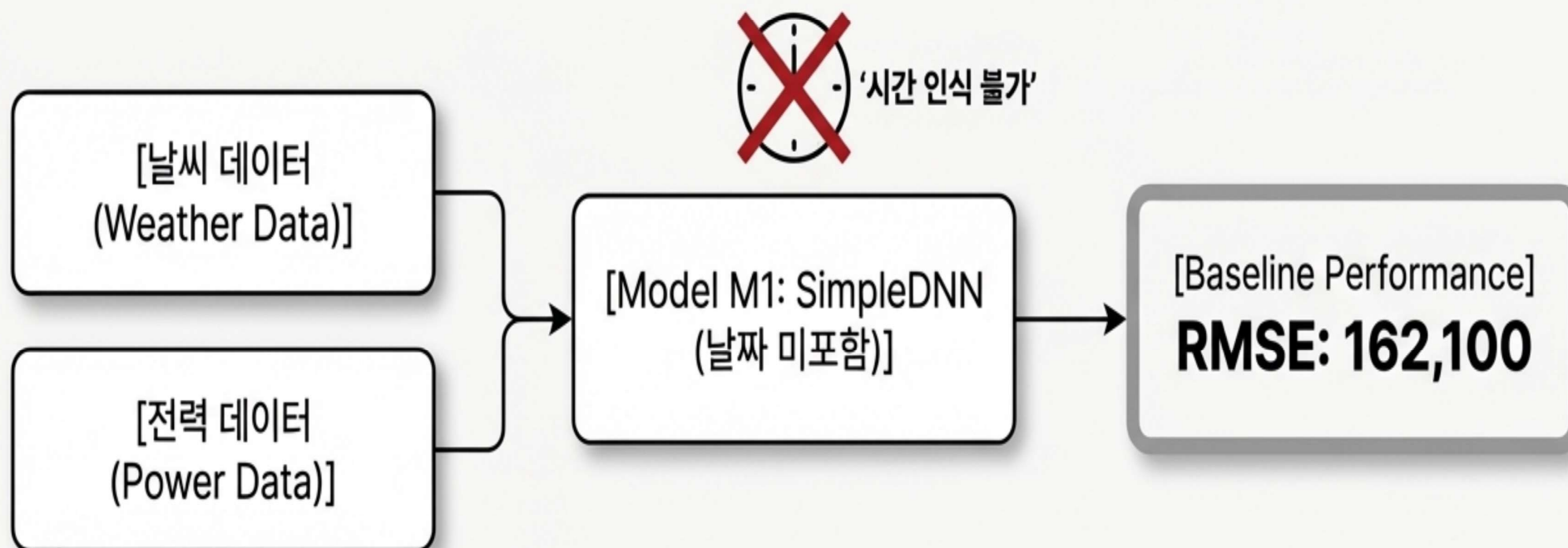
- 훈련에 사용될 최종 데이터셋

	지역코드	최저기온(°C)	0.5m 지중온도(°C)	평균 증기압(hPa)	가조시간(hr)	평균 상대습도(%)	파워
0	1111010100	-5.2	7.5	1.8	9.7	38.9	146294.6135
1	1111010100	-7.0	7.0	2.0	9.7	41.9	175633.8270
2	1111010100	-3.9	6.6	4.1	9.7	62.8	156084.1910
3	1111010100	0.1	6.6	4.4	9.7	57.5	177018.8420
4	1111010100	0.9	6.6	5.1	9.7	60.8	158467.1690

우리의 시작점: 시간 개념이 없는 Baseline 모델

현재 모델은 날씨, 전력 등 중요한 변수를 사용하지만, 시간에 따른 패턴을 인지하지 못합니다.
이 모델은 우리의 성능 비교를 위한 명확한 기준점(M1) 역할을 합니다.

중심 질문: '시간 정보를 추가하면 모델이 더 똑똑해질까?'



첫 번째 가설: 날짜 정보를 직접 주입하자

Experiment Design

Objective:

‘시간 정보를 넣었을 때, 모델이 실제로 더 잘 예측하는가?’를 검증하기 위한 명확한 성능 기준선 설정.

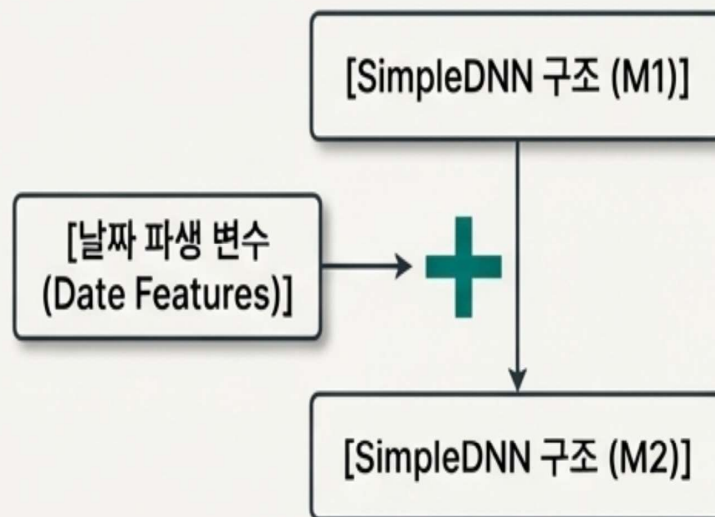
Methodology:

기존 Baseline 모델(SimpleDNN)과 동일한 구조 및 학습 조건에서, 입력 피처에 날짜 파생 변수만 추가.

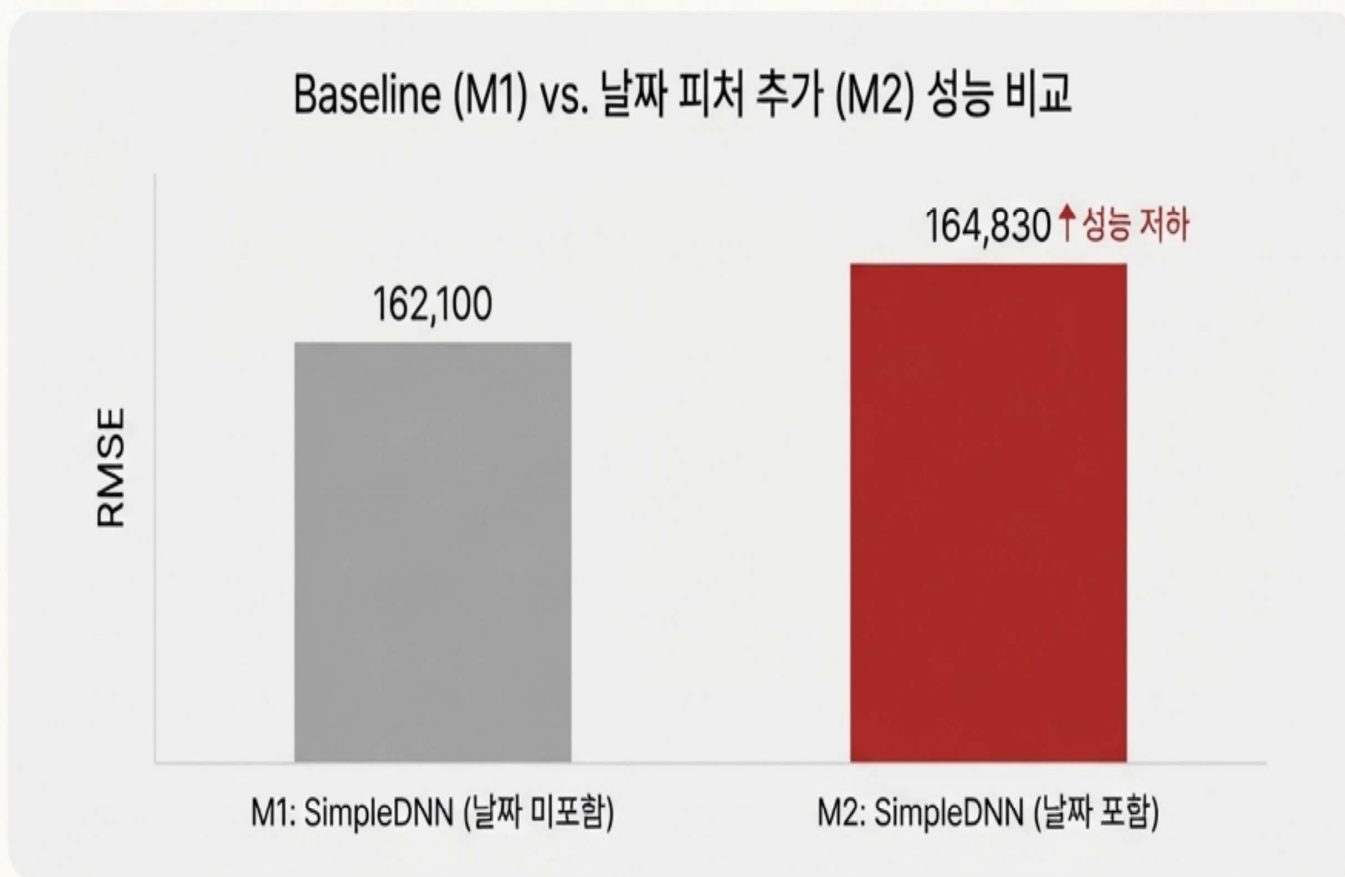
- 추가된 피처: 연, 월, 일, 요일 등 시간 기반 정보

Core Principle:

변수 하나(날짜)만 바꾼 통제된 비교 실험을 통해 시간 정보의 순수한 영향력을 측정.



예상 밖의 결과: 시간 정보가 오히려 성능을 저하시키다

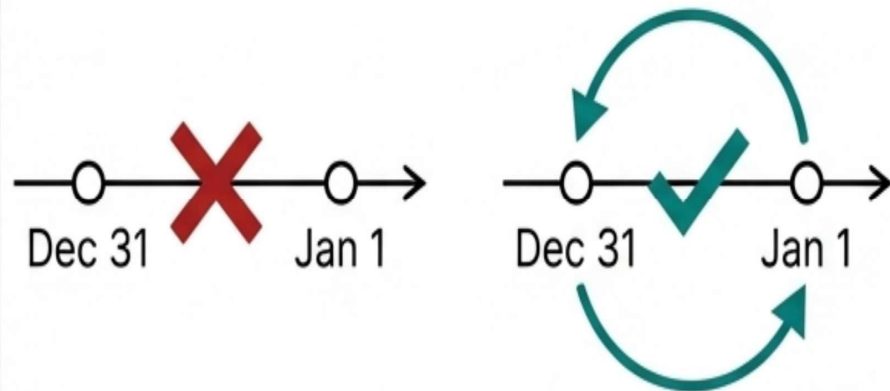


****Key Observation****: 날짜 관련 피처를 추가하자 모든 오차 지표(MSE, MAE, RMSE)가 소폭 상승했습니다. 단순한 시간 정보 주입은 효과적인 해결책이 아니었습니다.

무엇이 문제였을까? 단순 날짜 피처의 한계

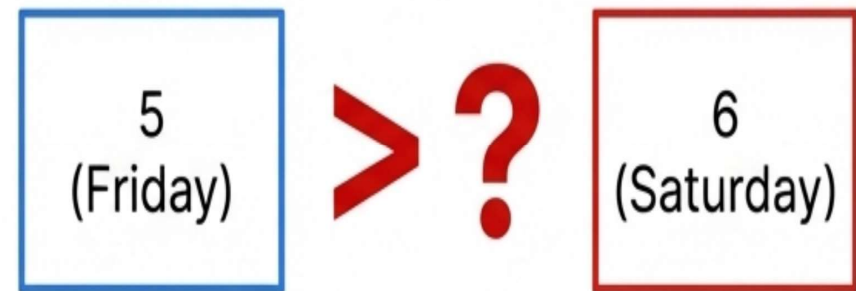
Hypothesis 1: Lack of Sequential Context

모델이 '연, 월, 일'을 독립적인 숫자로 인식하여, '12월 31일'과 '1월 1일' 사이의 순차적, 주기적 관계를 학습하지 못했을 가능성.



Hypothesis 2: Misleading Cardinality

요일과 같은 범주형 변수를 단순 숫자로 처리하면, 모델에게 의도치 않은 순서 관계(예: 토요일(6) > 금요일(5))를 학습시킬 수 있음.



Conclusion': 모델에게 '언제인지'를 알려주는 것보다, '직전에 어떤 일이 있었는지'를 알려주는 것이 더 효과적일 것이라는 새로운 가설을 세움.

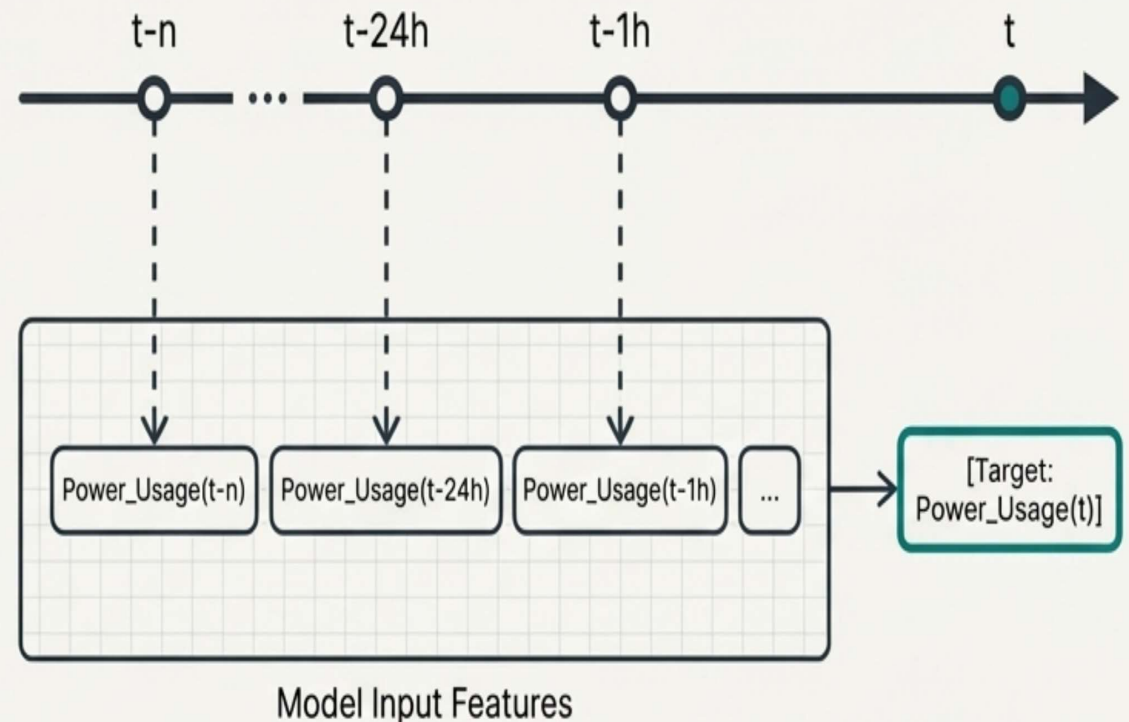
새로운 전략: 과거의 데이터로 현재를 설명하는 'Lag Feature'

What are Lag Features?

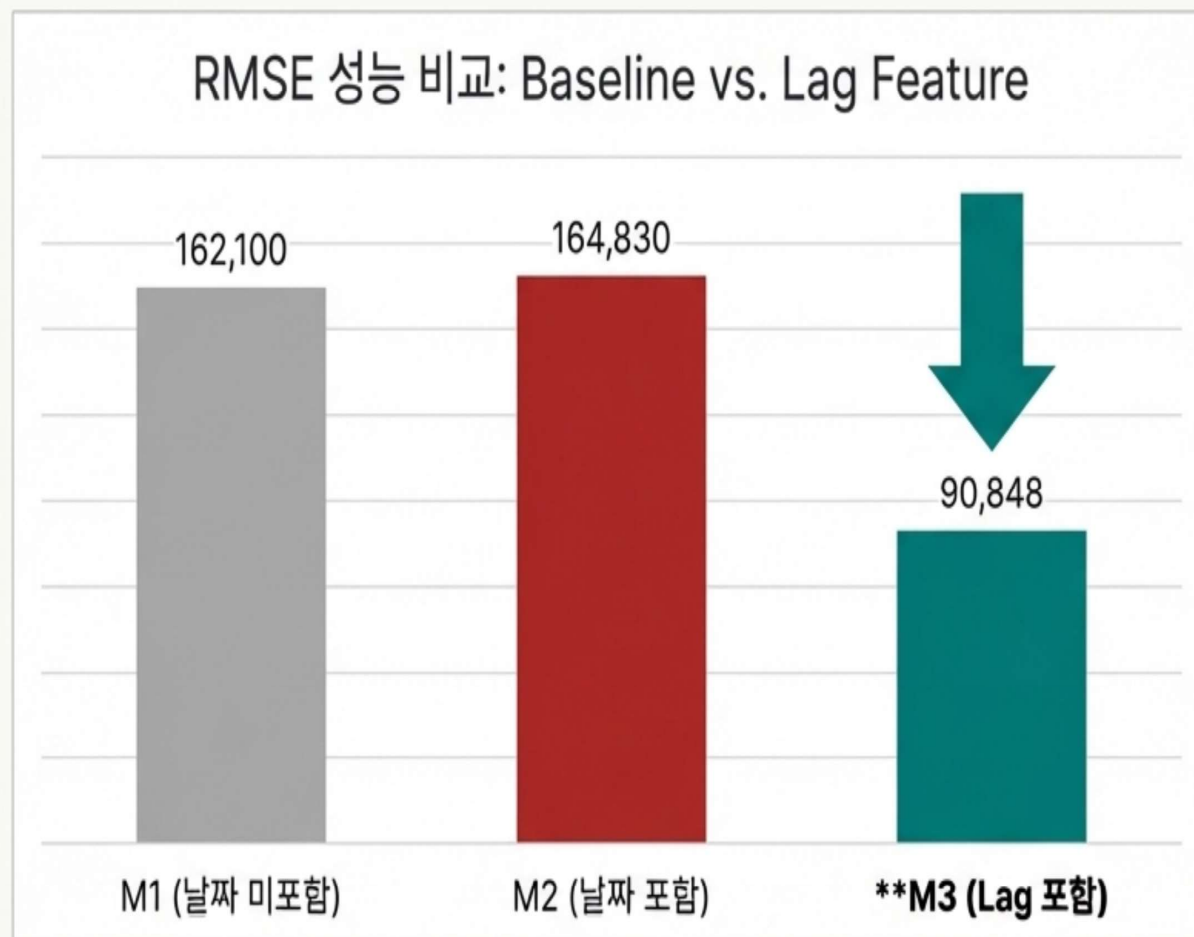
과거 시점의 데이터(예: 1일 전, 24시간 전, 1주일 전의 전력 사용량)를 현재 시점의 입력 피처로 사용하는 기법.

Why are they effective?

- **Direct Temporal Context:** 모델에게 데이터의 시간적 '흐름'과 '연속성'을 직접적으로 학습시킴.
- **Auto-correlation Capture:** "오늘 이 시간의 전력량은 어제 이 시간의 전력량과 비슷할 것이다"와 같은 강력한 자기 상관 관계를 명시적으로 모델링.



성능의 극적인 도약: Lag Feature의 압도적인 효과



Lag Feature를 도입한 SimpleDNN(M3) 모델의 RMSE가 기존 대비 약 44% 감소하며, 시간 정보 활용의 올바른 방향성을 증명했습니다.

최고의 피처 조합, 이제는 최고의 모델을 찾을 차례

우리는 가장 효과적인 시간 정보 피처(Lag)를 찾아냈습니다.
그렇다면 이 피처의 잠재력을 최대한 끌어낼 수 있는 모델 아키텍처는 무엇일까요?



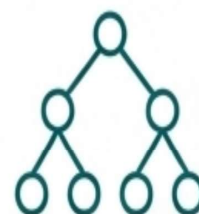
SimpleDNN (with Lag)

우리의 현재 챔피언.



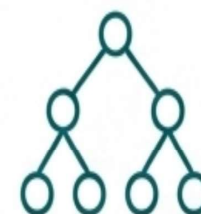
EmbeddingDNN

범주형 변수 처리에 강점을 가진 개선된 DNN.



LightGBM

Gradient Boosting 계열의 강력한 트리 기반 모델.



XGBoost

LightGBM과 쌍벽을 이루는 또 다른 트리 기반 모델.

Objective:: 동일한 피처셋(Lag)을 사용하여 각 모델의 성능을 정밀하게 비교 분석.

신경망 모델 비교: SimpleDNN vs EmbeddingDNN

SimpleDNN

- Fully-connected DNN 구조
- 연속형 변수 처리에 강점
- Lag Feature 추가 시 큰 성능 향상
- 단순하지만 효과적인 baseline
- 빠른 학습 속도
- 해석이 상대적으로 용이

EmbeddingDNN

- 임베딩 레이어 포함 DNN
- 범주형 변수의 패턴 자동 학습
- 날짜/요일 등 시간 정보 처리
- 더 깊은 네트워크 구조
- 복잡한 비선형 관계 포착
- 대규모 범주형 데이터에 강점

트리 기반 모델 비교: LightGBM vs XGBoost

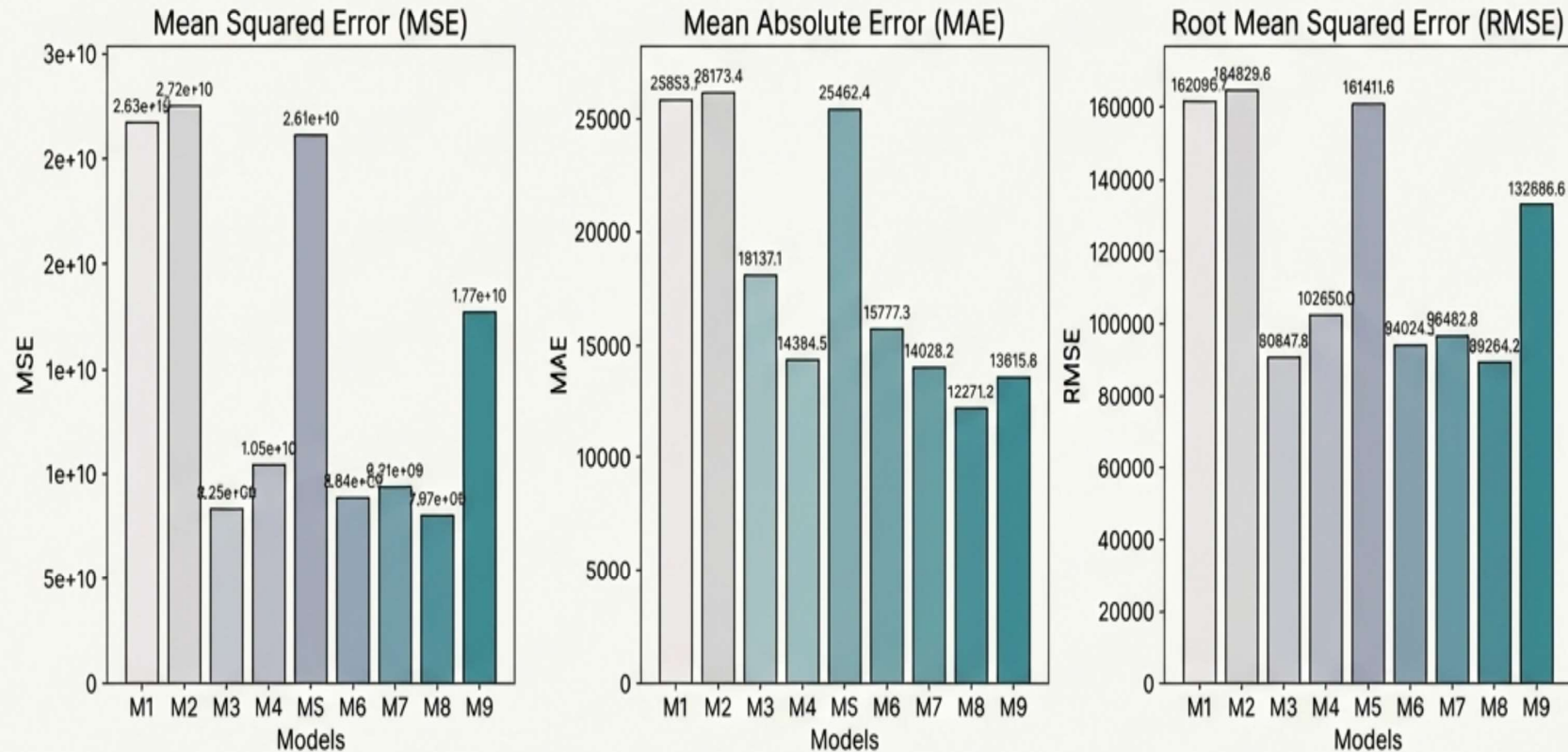
LightGBM

- Leaf-wise 성장 전략
- 더 빠른 학습 속도
- 메모리 효율적
- 범주형 변수 자동 처리
- 대용량 데이터 최적화
- 본 연구에서 최고 성능 달성

XGBoost

- Level-wise 성장 전략
- 높은 정확도
- 과적합 방지 우수
- 다양한 정규화 옵션
- 안정적 성능
- 산업 표준으로 널리 사용

최종 모델 성능 비교: DNN vs. Tree-Based Models



code	Model Name
M1	SimpleDNN (날짜 미포함)
M2	SimpleDNN (날짜 포함)
M3	SimpleDNN (LAG 포함)
M4	SimpleDNN (LAG_휴일 포함)
M5	EmbeddingDNN (LAG_휴일 포함)
M6	EmbeddingDNN (LAG_휴일 포함)
M7	EmbeddingDNN (LAG_휴일 포함)
M8	LIGHTGBM
M9	XGBoost

Lag Feature를 적용한 DNN 모델(M3)이 큰 성능 향상을 보였지만, 모든 평가 지표에서 트리 기반 모델, 특히 LightGBM(M8)이 가장 낮은 오차를 기록했습니다.

최종 승자: LightGBM

모든 평가 지표에서 LightGBM이 가장 낮은 오차를 기록하며,
현존하는 최적의 모델임이 입증되었습니다.

Model Name	MSE	MAE	RMSE
SimpleDNN (LAG 포함) - M3	8.25e+09	18,137	90,848
LightGBM - M8	7.97e+09	12,271	89,264
EmbeddingDNN (Best) - M6	8.83e+09	15,777	93,998
XGBoost - M9	1.77e+10	13,616	132,887

왜 LightGBM이 최고 성능을 달성했는가?

LightGBM (M8): RMSE 89,264 - 전체 모델 중 최저 오차

1 Leaf-wise 성장 전략

복잡한 비선형 패턴을
효과적으로 학습하며
시간적 종속성을 정확히 포착

2 Lag Feature와의 시너지

과거 전력 사용량 데이터의
중요도를 정확히 파악하고
시계열 패턴 학습에 최적화

3 범주형 변수 최적 처리

날짜 파생 변수를
자동으로 최적화하여
효율적인 분할 수행

4 학습 속도와 정확도

빠른 학습 속도로
여러 실험이 가능하며
높은 정확도 보장

이번 분석을 통해 얻은 핵심 교훈



Learning 1: Feature Engineering is Crucial

어떻게 시간 정보를 표현하는가(How)가 단순히 추가하는 것(Whether)보다 훨씬 중요합니다. 순진한(naive) 피처링은 오히려 해가 될 수 있습니다.



Learning 2: Lag Features are King

본 문제에서 시간적 종속성을 포착하는 가장 효과적인 방법은 Lag Feature였습니다.



Learning 3: Test Everything

최고의 피처셋과 최고의 모델 아키텍처는 별개의 문제입니다. 두 가지를 모두 실험하고 최적의 조합을 찾는 체계적인 접근이 필수적입니다. LightGBM은 이 조합에서 승리했습니다.

최종 제언 및 향후 계획

Immediate Recommendation

'Lag Feature + LightGBM' 조합을 새로운 Baseline 모델로 채택하여 운영 시스템에 적용할 것을 제안합니다.

Future Work

1. **Hyperparameter Optimization:** 선정된 LightGBM 모델의 성능을 극대화하기 위한 정교한 하이퍼파라미터 튜닝을 진행합니다.
2. **Advanced Feature Exploration:** 이동 평균(Moving Average), 계절성(Seasonality) 등 더 복잡한 시계열 피처를 추가하여 추가 성능 향상 가능성을 탐색합니다.
3. **Ensemble Models:** LightGBM과 다른 모델(e.g., DNN)의 예측을 결합하는 앙상블 기법을 테스트합니다.

Appendix: Detailed Model Performance Metrics

	Model Name	MSE	MAE	RMSE
M1	SimpleDNN (날짜 미포함)	26276315136	25853.3457	162099.7074
M2	SimpleDNN (날짜 포함)	27168802816	26173.35156	164829.6175
M3	SimpleDNN (LAG 포함)	8253324288	18137.05469	90847.80838
M4	SimpleDNN (LAG_휴일 포함)	10537024512	14394.45898	102650.0098
M5	EmbeddingDNN (LAG_휴일 포함)	26053705728	25462.42188	161411.6034
M6	EmbeddingDNN (LAG_휴일 포함)	8835653632	15777.30762	93998.1576
M7	EmbeddingDNN (LAG_휴일 포함)	9311087616	14028.22656	96493.9771
M8	LightGBM	7968094463	12271.23464	89264.18354
M9	XGBoost	17658850371	13615.59043	132886.6072

Appendix : Flask Webpage

The screenshot shows a web application interface for AI demand forecasting. At the top, a blue banner contains the title '전 수요 예측 모델 분석 대시보드' (AI Demand Forecasting Model Analysis Dashboard) and a subtitle '기상·환경 데이터를 기반으로 AI 전력 수요를 예측하고, DNN / XGBoost 모델 성능을 비교합니다.' (Predict AI electricity demand based on weather/environmental data and compare DNN / XGBoost model performance). Below the banner are four navigation buttons: '예측 바로 해보기 ↓' (Predict Now), '모델 성능 보기' (View Model Performance), '전처리 과정' (Preprocessing Process), and 'EDA 분석' (EDA Analysis). The main content area is divided into two sections. The left section, 'AI 수요 예측 시뮬레이터' (AI Demand Forecasting Simulator), includes a description of input methods, a toggle for '날짜 기준 예측' (Date-based Prediction), and a 'STEP 1 · 대상 지역 선택' (STEP 1 · Select Target Area) section with dropdown menus for '구 선택' (Select District) and '동 선택' (Select Dong). The right section, 'DATE-BASED', is currently '대기중' (Waiting). Navigation arrows on the right side of the page point to the corresponding sections: '① 예측 바로 해보기 → 예측 시뮬레이터 섹션으로 이동' (Click Predict Now to move to the simulation section), '② 모델 성능 보기 → 모델 성능 비교 그래프 이동' (Click View Model Performance to move to the comparison graph), '③ 전처리 과정 → 데이터 전처리 및 피처 엔지니어링 설명' (Click Preprocessing Process to view the explanation), and '④ EDA 분석 → 데이터 탐색 분석 결과 이동' (Click EDA Analysis to move to the results).

전 수요 예측 모델 분석 대시보드

기상·환경 데이터를 기반으로 AI 전력 수요를 예측하고, DNN / XGBoost 모델 성능을 비교합니다.

[예측 바로 해보기 ↓](#) [모델 성능 보기](#) [전처리 과정](#) [EDA 분석](#)

① 예측 바로 해보기
→ 예측 시뮬레이터 섹션으로 이동

② 모델 성능 보기
→ 모델 성능 비교 그래프 이동

③ 전처리 과정
→ 데이터 전처리 및 피처 엔지니어링 설명

④ EDA 분석
→ 데이터 탐색 분석 결과 이동

AI 수요 예측 시뮬레이터

입력 방식(탭)을 선택하고 분석을 실행하면, 예측값과 간단한 해석 지표를 함께 제공합니다.

환경 변수(시뮬레이션) 날짜 기준 예측

● STEP 1 · 대상 지역 선택

구 선택
-- 구를 선택하세요 --

동 선택
-- 먼저 구를 선택하세요 --

● DATE-BASED 대기중

Appendix : Flask Webpage

AI 수요 예측 시뮬레이터

입력 방식(탭)을 선택하고 분석을 실행하면, 예측값과 간단한 해석 지표를 함께 제공합니다.

환경 변수(시뮬레이션)

날짜 기준 예측

STEP 1 · 대상 지역 선택

구 선택

-- 구를 선택하세요 --

동 선택

-- 먼저 구를 선택하세요 --

STEP 2 · 입력 방식

최저기온 (°C)

22.5

지중온도 (3m, °C)

18.2

상대습도 (%)

65

현지기압 (hPa)

1013.2

가조시간 (hr)

14.5

특이사항

평일

1일전 전력사용량(KWH)

5000

7일전 전력사용량(KWH)

5000

* 환경 변수를 직접 입력해 “가정 시뮬레이션”으로 예측을 비교할 수 있어요.

분석 및 예측 실행

① 입력 방식 선택: 환경 변수(시뮬레이션)
→ 사용자가 직접 조건을 조정해 가상 시나리오 예측

② STEP 1. 대상 지역 선택
→ 지역별 전력 사용 패턴 반영

③ STEP 2. 환경 변수 입력
→ 기온·습도·기압 등 직접 설정

예측 결과 패널

탭에서 입력 방식을 고르고 분석 및 예측 실행을 눌러보세요.

Tip: “환경 변수” 탭에서 휴일/평일을 바꾸면 결과 차이가 확 보여요.

⑤ 분석 및 예측 실행
→ 가상 시나리오 기반 전력 수요 예측

④ 과거 전력 사용량 입력
→ 단기 시계열 패턴 반영

Appendix : Flask Webpage

AI 수요 예측 시뮬레이터

입력 방식(탭)을 선택하고 분석을 실행하면, 예측값과 간단한 해석 지표를 함께 제공합니다.

환경 변수(시뮬레이션)

날짜 기준 예측

● STEP 1 · 대상 지역 선택

구 선택

-- 구를 선택하세요 --

동 선택

-- 먼저 구를 선택하세요 --

● STEP 2 · 입력 방식

예측 대상 날짜

2026-01-05

* 선택한 날짜의 공공 기상 데이터 API 수치를 기반으로 자동 예측(데모)합니다.

분석 및 예측 실행

① 입력 방식 선택
→ 환경 변수 / 날짜 기준 전환

● DATE-BASED

● 대기중

② 지역 선택
→ 예측 대상 지역 지정

⑤ 결과 확인
→ 예측 전력 수요 출력

③ 기준 날짜 선택
→ 기상 데이터 자동 반영

예측 결과 패널

Tip: "환경 변수" 탭에서 휴일/평일을 바꾸면 결과 차이가 확 보여요.

④ 분석 및 예측 실행
→ 모델 추론 수행